DATA 412/612 · Statistical Programming in R · Week 8

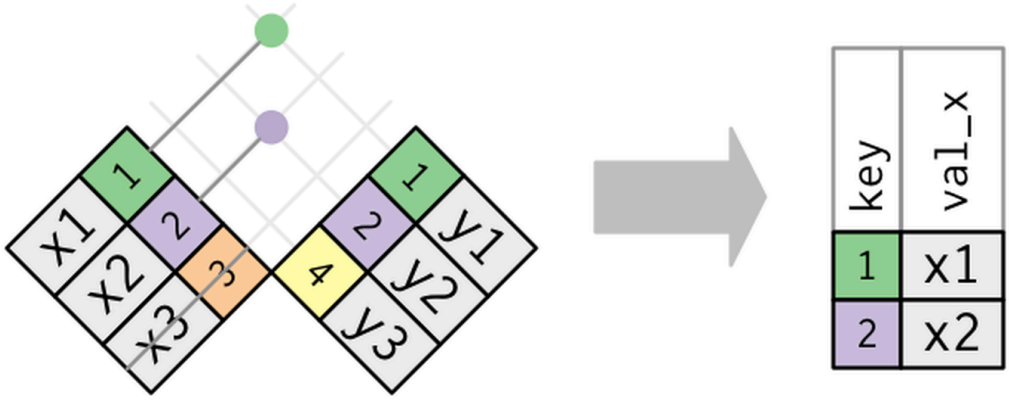
8.2 dplyr: Filtering Joins

semi_join() and anti_join() keep or drop rows of x. They do not add columns.

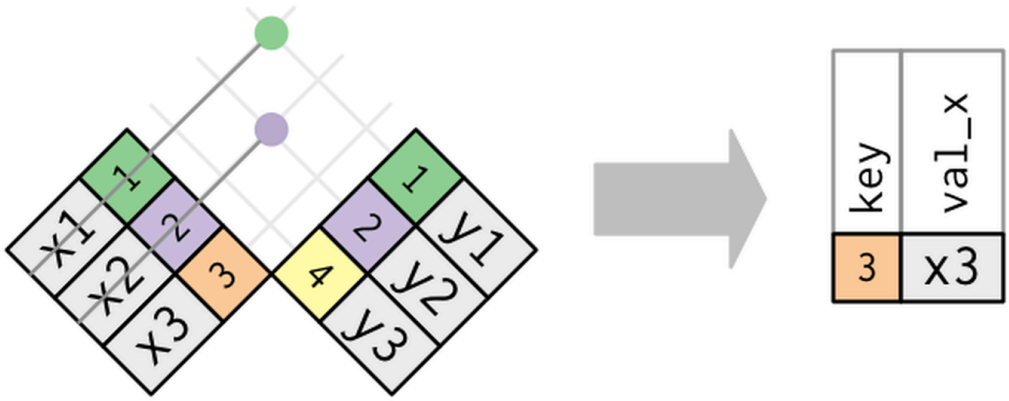
1. `semi_join()` and `anti_join()`

A mutating join **adds columns**. A filtering join **keeps or drops rows** of `x` and leaves the columns of `x` alone.

- `semi_join(x, y)` — rows of `x` that have a match in `y`



- `anti_join(x, y)` — rows of `x` that do **not** have a match in `y`




```
library(tidyverse)
library(nycflights13)

flights |>
  semi_join(planes, by = "tailnum")

flights |>
  anti_join(planes, by = "tailnum")
```

`inner_join(x, y)` also drops unmatched rows of `x`, but it **duplicates** a row of `x` once for each match in `y`. `semi_join()` never duplicates rows of `x`. When the question is only whether a match exists, use `semi_join()`.

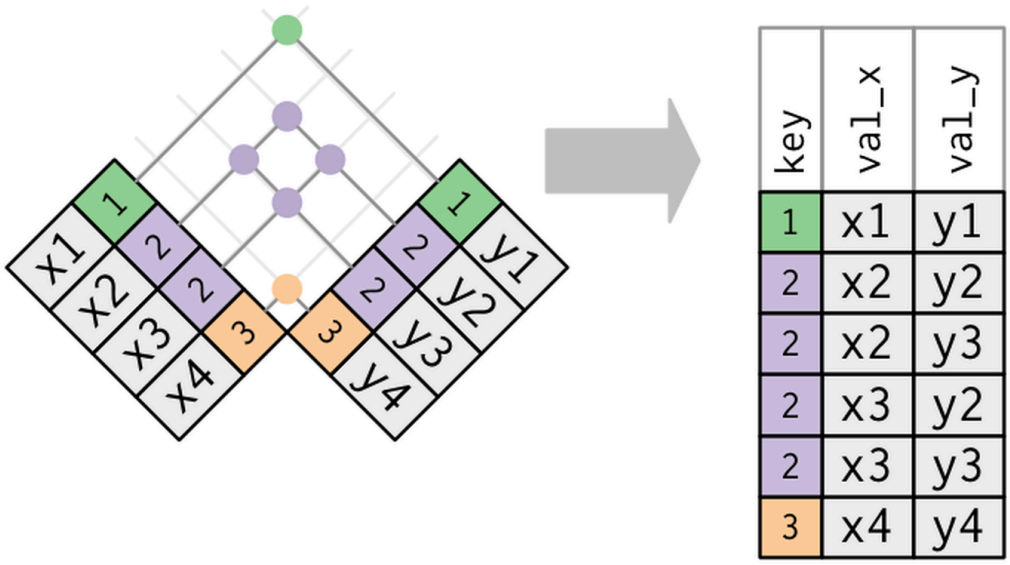
`anti_join()` identifies the unmatched rows: destinations with no airport record, aircraft that never flew, and weather hours with no flights.

```
flights |>
  anti_join(airports, by = c("dest" = "faa")) |>
  distinct(dest)
```

---

2. Duplicate keys and many-to-many

If a key appears twice in `x` and three times in `y`, an inner or left join returns six rows for that key. The result is a many-to-many join. The expansion is sometimes the intended result. It often indicates that the key does not identify rows uniquely.




```
x <- tribble(
  ~key, ~val_x,
     1, "x1",
     1, "x2"
)
y <- tribble(
  ~key, ~val_y,
     1, "y1",
     1, "y2"
)

left_join(x, y, by = "key")
semi_join(x, y, by = "key")
```

The left join has four rows. The semi join still has two — the original `x`. Check keys with `count()` before you join (note 8.1).

---

3. Worst delay days, then those flights

Ten days with the highest median `dep_delay`, then every flight on those days. Build a small table of keys, then `semi_join()`.

```
worst <- flights |>
  group_by(year, month, day) |>
  summarize(med_del = median(dep_delay, na.rm = TRUE), .groups = "drop") |>
  slice_max(med_del, n = 10)

flights |>
  semi_join(worst, by = c("year", "month", "day"))
```

`filter()` with a long `%in%` list is clumsier once the key has several columns.

---

4. Plane age and delay

`planes$year` is the year of manufacture. Age in 2013 is `2013 - year`. Join, then plot. Rename `year` first so it does not clash with `flights$year`.

```
flights |>
  left_join(
    planes |> select(tailnum, plane_year = year),
    by = "tailnum"
  ) |>
  mutate(age = 2013 - plane_year) |>
  filter(!is.na(age), !is.na(dep_delay)) |>
  ggplot(aes(x = age, y = dep_delay)) +
  geom_hex() +
  geom_smooth(se = FALSE) +
  theme_bw()
```

`geom_hex()` needs the `hexbin` package. `geom_smooth()` alone is enough if you do not want the bins.

---

5. Practice

1. Destinations in `flights` that are missing from `airports`. How many flights is that?
2. Airports that never appear as a `dest`.
3. The ten days with the highest median `dep_delay`, then all flights on those days.
4. A plot of plane age against `dep_delay`.